

Entrevista Técnica — Oficina MotorOK (desenhe no draw.io)

Como funciona este exercício. Abaixo está uma **conversa** entre a equipe e o cliente do sistema **Oficina MotorOK** (dono de uma oficina mecânica). **Leia com atenção** e, a partir do que as pessoas dizem,

desenhe um diagrama de classes no draw.io (ou app.diagrams.net).

Não é para programar — é para **modelar**.

 **Seu diagrama precisa mostrar os quatro pilares da OO e os três tipos de relacionamento** (associação, agregação e composição). No fim há a **lista do que entregar**, um **lembrete da notação** e um **gabarito** (para o professor).

A entrevista

Gerente: Pessoal, vamos organizar o sistema da **Oficina MotorOK**. Vou trazer o cliente pra ele explicar, do jeito dele, o que o sistema precisa ter. Anotem tudo e depois a gente transforma em diagrama.

Cliente: No sistema **Oficina MotorOK**, a peça central é **Cliente**. Tem dois tipos: o

ClienteComum (o comum) e o **ClienteFrota**, que **tem tabela de preço própria e prazo diferenciado**.

No fundo, os dois são um tipo de

Cliente — todos têm nome e documento e compartilham a mesma base.

Dev sênior: Entendi. Então tem uma parte **igual pra todos** e uma parte **específica** de cada tipo. Guardem isso — cheira a uma base comum com dois tipos que herdam dela.

Cliente: Isso. E o comportamento muda por tipo: olhando o `precoMaoDeObra()` de cada um, pro

ClienteComum paga a tabela padrão; já pro **ClienteFrota** paga a tabela da frota (contrato). A **mesma** operação, **resposta**

diferente conforme o tipo.

Analista de qualidade: Um ponto sério: o valor do item não pode ser mexido por fora (nunca negativo).

Ninguém pode mexer nesses dados **por fora** —

tem que ser por **operações controladas**. E tem uma regra que **NÃO pode falhar: não pode executar a OS antes de o orçamento ser aprovado**.

Por isso o estado de **OrdemServico** só anda por operações (aberta → aprovada → em execução → concluída → paga), nunca "na mão".

Cliente: Agora as ligações. Um cliente tem VÁRIAS ordens de serviço; cada OS é de UM veículo.

Cliente: Tem uma ligação de **parte-todo que nasce e morre junto: OrdemServico ◆ ItemOS**.

Cada item (serviço/peça) da OS só existe dentro dela e some junto se a OS for cancelada. (pense em **composição** — losango cheio ◆)

Dev sênior: Não confunda com o outro caso: **CatalogoServicos** ◇ **Servico** é só um **agrupamento**. Os serviços existem no catálogo por conta própria; a OS só os referencia. Se **CatalogoServicos** sumir, **Servico** continua existindo — isso é **agregação** (losango vazio ◇), bem diferente da composição.

Gerente: Fechou. Com isso dá pra desenhar o modelo. Mãos à obra.

Dicas de leitura (pistas escondidas na conversa)

O que apareceu na conversa	Onde isso te leva
"os dois são um tipo de Cliente (mesma base)"	um tipo geral (base) e tipos específicos → herança
"a mesma operação precoMaoDeObra() , resposta muda por tipo"	operação redefinida em cada tipo → polimorfismo
"O valor do item não pode ser mexido por fora (nunca negativo)"	dado privado + operações → encapsulamento
"não pode executar a OS antes de o orçamento ser aprovado"	invariante : o estado muda só por operação que valida
" OrdemServico ◆ ItemOS (nascem e somem juntos)"	composição (losango cheio ◆)
" CatalogoServicos ◇ Servico (Servico existe sozinho)"	agregação (losango vazio ◇)
"um cliente tem VÁRIAS ordens de serviço; cada OS é de UM veículo"	associação com multiplicidade

O que você deve entregar (no draw.io)

Um **diagrama de classes** que contenha, de forma clara:

1. **Abstração** — uma classe **base** que não é criada diretamente (ex.: **Cliente**).
2. **Herança** — **ClienteComum** e **ClienteFrota** ligados à base pelo triângulo vazio →▷.
3. **Encapsulamento** — atributos sensíveis como **privados** (−) e **operações públicas** (+) que os controlam (proteger o valor e o estado da **OrdemServico**).
4. **Polimorfismo** — a operação **precoMaoDeObra()** declarada na base e **redefinida** em cada tipo.
5. **Os três relacionamentos**, cada um com sua **multiplicidade**:
 - **Associação** (linha/seta): um cliente tem VÁRIAS ordens de serviço; cada OS é de UM veículo.
 - **Agregação** (losango **vazio** ◇): **CatalogoServicos** junta **Servicos** que existem sozinhos.
 - **Composição** (losango **cheio** ◆): **OrdemServico** é feita de **ItemOSs** que nascem e morrem com ela.

Lembrete de notação (UML)

Elemento	Como desenhar
Classe abstrata	nome em <i>itálico</i> + « abstract » (não se cria direto)
Operação abstrata	em <i>itálico</i> (cada subtipo redefine)
Visibilidade	– privado · + público · # protegido
Herança	linha com triângulo vazio → apontando para a base
Associação	linha (com seta), com multiplicidade nas pontas
Agregação	losango vazio ◇ no lado do "todo"
Composição	losango cheio ◆ no lado do "todo"

Multiplicidades: **1** (exatamente um), **0..1** (zero ou um), ***** (muitos), **1..*** (um ou mais).

Critério de "pronto"

- ☐ Há uma **classe base abstrata** e os dois tipos herdando dela.
- ☐ Nenhum atributo sensível está público; o valor e o estado da **OrdemServico** são protegidos.
- ☐ A operação **precoMaoDeObra()** aparece na base e é **redefinida** em cada tipo.
- ☐ Aparecem os **três** relacionamentos: **associação**, **agregação** (◇) e **composição** (◆).
- ☐ Você consegue **explicar** por que **OrdemServico** → **ItemOS** é composição e **CatalogoServicos** → **Servico** é agregação.
- ☐ Todas as ligações têm **multiplicidade**.

 [Briefing do projeto](#) ·  [Plano de evolução](#)